

Entendido. Ya tienes el database link `@mssqlldb` funcionando desde Oracle hacia SQL Server. El objetivo es replicar la lógica de bloqueo de puntos de venta en el lado de SQL Server para que el sistema que consume esa base de datos también impida emitir ventas. La solución es directa y no requiere software adicional.

A continuación, tienes la guía completa con los tres componentes necesarios: la tabla espejo en SQL Server, el proceso de sincronización desde Oracle y el trigger de validación en SQL Server.

1. Crear la tabla de bloqueos en SQL Server

Primero, crea la tabla `tbbloqueopv` en la base de datos de SQL Server. Esta tabla recibirá la lista de puntos de venta bloqueados desde Oracle.

sql

```
-- Ejecutar en SQL Server (en la base de datos del sistema de ventas)
```

```
CREATE TABLE dbo.tbbloqueopv (  
    id_punto_venta    INT NOT NULL PRIMARY KEY,  
    motivo            VARCHAR(200) NULL,  
    fecha_bloqueo     DATETIME NOT NULL DEFAULT GETDATE()  
);
```

- `id_punto_venta`: Clave primaria que identifica el punto de venta. Debe coincidir con el `id_punto_venta` de Oracle.
- `motivo`: Campo opcional para guardar la razón del bloqueo (ej. "Mora en facturas").
- `fecha_bloqueo`: Fecha en que se sincronizó el bloqueo.

Nota: Si ya existe una tabla similar, asegúrate de que los nombres de las columnas coincidan con los que usarás en el `MERGE` del paso 2.

2. Procedimiento de sincronización en Oracle (a través del database link)

Crea un procedimiento en Oracle que lea los puntos de venta bloqueados de tu tabla local `tb_bloq_p_v` y los inserte o actualice en la tabla `tbbloqueopv` de SQL Server usando el database link `@mssqlldb`.

sql

```
CREATE OR REPLACE PROCEDURE PR_SINC_BLOQUEOS_A_SQL AS
BEGIN
    -- MERGE a través del database Link
    MERGE INTO "tbbloqueopv"@mssqlldb t
    USING (
        SELECT id_punto_venta,
               motivo,
               fecha_bloqueo
        FROM tb_bloq_p_v
        WHERE estado = 'INACTIVO' -- Solo los que están bloqueados
    ) s
    ON (t.id_punto_venta = s.id_punto_venta)
    WHEN MATCHED THEN
        UPDATE SET t.motivo = s.motivo,
                  t.fecha_bloqueo = SYSDATE
    WHEN NOT MATCHED THEN
        INSERT (id_punto_venta, motivo, fecha_bloqueo)
        VALUES (s.id_punto_venta, s.motivo, SYSDATE);

    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        -- Registrar el error para diagnóstico
        DBMS_OUTPUT.PUT_LINE('Error en sincronización: ' || SQLERRM);
        RAISE;
END;
/
```

Consideraciones sobre el MERGE a través del database link:

- El `MERGE` es una sentencia DML que Oracle puede enviar al gateway de SQL Server. El gateway traduce la operación a un `MERGE` de SQL Server o a un `UPDATE/INSERT` equivalente.
 - La sintaxis `MERGE INTO tabla@link` es soportada por el Oracle Database Gateway para SQL Server.
 - Si el gateway no soporta `MERGE` directamente, el optimizador de Oracle puede descomponerlo en operaciones separadas. En la práctica, funciona correctamente para volúmenes moderados.
-

3. Programar el Job en Oracle

El procedimiento anterior debe ejecutarse **inmediatamente después** de tu proceso de bloqueo automático (el que corre cada 2 horas). Tienes dos opciones:

Opción A: Añadir la llamada al final del procedimiento de bloqueo existente

Si tu procedimiento de bloqueo automático se llama `PR_BLOQUEO_AUTOMATICO_PV`, simplemente añade al final:

sql

```
-- Dentro de PR_BLOQUEO_AUTOMATICO_PV, después del COMMIT  
PR_SINC_BLOQUEOS_A_SQL;
```

Opción B: Crear un Job independiente que se ejecute con un pequeño retraso

Si prefieres mantener los procesos separados, programa un Job que se ejecute 5 minutos después de cada ejecución del bloqueo. Por ejemplo, si el bloqueo corre a las 6:00, 8:00, 10:00..., el job de sincronización puede correr a las 6:05, 8:05, 10:05...

sql

```
BEGIN
```

```

DBMS_SCHEDULER.CREATE_JOB (
    job_name          => 'JOB_SINC_BLOQUEOS_SQL',
    job_type          => 'STORED_PROCEDURE',
    job_action         => 'PR_SINC_BLOQUEOS_A_SQL',
    start_date         => TRUNC(SYSDATE) + 6/24 + 5/1440, -- Hoy a Las 6:05 AM
    repeat_interval    => 'FREQ=HOURLY; INTERVAL=2;
BYHOUR=6,8,10,12,14,16,18,20; BYMINUTE=5',
    enabled            => TRUE
);
END;
/

```

Importante: Si el proceso de bloqueo se ejecuta a las 6:00, 8:00, etc., el job de sincronización debe ejecutarse **después** de que el bloqueo haya terminado y haya hecho COMMIT. Un retraso de 5 minutos es suficiente para asegurar que los datos estén disponibles.

4. Crear el trigger de validación en SQL Server

Ahora, en la tabla de ventas de SQL Server (supongamos que se llama `ventas`), crea un trigger `INSTEAD OF INSERT` que verifique si el punto de venta está en la tabla `tbbloqueopv`. Si lo está, lanza un error con `THROW` y no permite la inserción.

sql

```

-- Ejecutar en SQL Server
CREATE TRIGGER trg_validar_bloqueo_pv
ON dbo.ventas
INSTEAD OF INSERT
AS
BEGIN
    SET NOCOUNT ON;

    -- Verificar si alguno de los puntos de venta insertados está bloqueado
    IF EXISTS (
        SELECT 1
        FROM inserted i
        JOIN dbo.tbbloqueopv b ON b.id_punto_venta = i.id_punto_venta
    )

```

```

BEGIN
    -- Lanzar un error personalizado. El número 50000 es un mensaje
    definido por el usuario.
    THROW 50000, 'El punto de venta está bloqueado. No se puede emitir la
    venta.', 1;
END

-- Si no hay bloqueos, realizar la inserción real
INSERT INTO dbo.ventas (id_venta, id_punto_venta, fecha_venta, monto)
SELECT id_venta, id_punto_venta, fecha_venta, monto
FROM inserted;
END;

```

Puntos clave del trigger:

- **INSTEAD OF INSERT:** Se ejecuta **antes** de que la inserción se realice realmente. Esto te permite validar y, opcionalmente, realizar la inserción tú mismo.
- **inserted:** Es una tabla temporal que contiene las filas que se están intentando insertar. Puede contener **una o varias filas**, por eso se usa **EXISTS** en lugar de **SELECT TOP 1**.
- **THROW:** Lanza una excepción que la aplicación cliente (el sistema de ventas) capturará como un error. El mensaje es claro y directo para el usuario final.
- **La inserción real:** Si no hay bloqueos, el trigger realiza la inserción en la tabla **ventas** usando **SELECT ... FROM inserted**.

Alternativa con RAISERROR: Si tu versión de SQL Server es anterior a 2012, usa **RAISERROR('El punto de venta está bloqueado...', 16, 1)** en lugar de **THROW**. Sin embargo, **THROW** es la opción recomendada y moderna.

5. Flujo completo del proceso

1. **Cada 2 horas**, el Job en Oracle ejecuta **PR_BLOQUEO_AUTOMATICO_PV** y bloquea los puntos de venta morosos en **tb_bloq_p_v**.
2. **Inmediatamente después** (o 5 minutos después), el Job **JOB_SINC_BLOQUEOS_SQL** ejecuta **PR_SINC_BLOQUEOS_A_SQL**, que hace un **MERGE** a través del database link **@mssqldb** para insertar o actualizar los registros en **tbbloqueopv** de SQL Server.

3. Cuando el sistema de ventas (en SQL Server) intenta insertar una nueva venta en la tabla `ventas`, el trigger `trg_validar_bloqueo_pv` se dispara.
 4. El trigger verifica si el `id_punto_venta` está en `tbbloqueopv`. Si está, lanza un error con `THROW` y la venta **no se inserta**.
 5. La aplicación que usa SQL Server recibe el mensaje de error y se lo muestra al usuario.
-

6. Consideraciones finales

- **Rendimiento:** El `MERGE` a través del database link puede ser más lento que una operación local, pero para el volumen de puntos de venta que se bloquean cada 2 horas (normalmente decenas o cientos), es perfectamente aceptable.
- **Atomicidad:** El `MERGE` en Oracle se ejecuta como una transacción distribuida. Si falla a mitad de camino, el gateway maneja el rollback. Siempre haz `COMMIT` después del `MERGE` en Oracle para confirmar los cambios en SQL Server.
- **Manejo de errores:** Envuelve el `MERGE` en un bloque `BEGIN ... EXCEPTION ... END` para capturar errores de red o de conexión con el gateway. Si falla, el Job de Oracle debe reintentar en la próxima ejecución.
- **Pruebas:** Antes de poner esto en producción, prueba el flujo completo en un entorno de desarrollo:
 1. Bloquea manualmente un PV en Oracle.
 2. Ejecuta `PR_SINC_BLOQUEOS_A_SQL`.
 3. Verifica que el registro aparezca en `tbbloqueopv` de SQL Server.
 4. Intenta insertar una venta para ese PV en SQL Server.
 5. Confirma que el trigger lanza el error y la venta no se inserta.

Con esta implementación, tendrás la sincronización bidireccional de bloqueos funcionando sin necesidad de software adicional, solo con las herramientas nativas de Oracle y SQL Server.

en lugar del merge dame el codigo scipr con el update y el insert, sin usar merge para evitar problemas. dame el codigo completo de todo el procedimiento, y dime si puedo crear una tabla y el trigger desde oracle con el link que ya existe a sql server

El usuario quiere reemplazar el `MERGE` con `UPDATE` e `INSERT` separados en el procedimiento de sincronización de Oracle a SQL Server, y también pregunta si puede crear la tabla y el

trigger en SQL Server directamente desde Oracle usando el database link existente. Necesito proporcionar el código completo del procedimiento con UPDATE e INSERT, y una respuesta clara sobre las posibilidades de DDL a través del link.

Para abordar esto, necesito buscar información sobre:

1. Cómo ejecutar DDL (CREATE TABLE, CREATE TRIGGER) en SQL Server a través de un database link de Oracle.
2. La sintaxis de UPDATE e INSERT a través de un database link de Oracle.
3. Consideraciones sobre el uso de DBMS_HS_PASSTHROUGH para DDL.

Voy a realizar búsquedas en español para obtener información relevante.

Found 36 web pages

Los resultados de búsqueda proporcionan información sobre DDL a través de database links. El resultado 0 de la búsqueda 0 menciona `dbms_utility.exec_ddl_statement` y `dbms_utility.exec_ddl_statement@dblink`. El resultado 1 de la búsqueda 1 menciona `DBMS_HS_PASSTHROUGH.EXECUTE_IMMEDIATE`. El resultado 0 de la búsqueda 3 indica que no se puede crear un trigger en una base de datos remota. Necesito abrir algunos de estos resultados para obtener más detalles.

Read 4 pages

[How to create a table from remote database using dblink - How to create a table from remote database using dblink - Stack Overflow](#)
[most recent 30 from stackoverflow](#)

[Další možností přístupu do MSSQL z Oracle je využití balíčku DBMS_HS_PASSTHROUGH a příkazu EXECUTE IMMEDIATE](#)

[Use DBMS_HS_PASSTHROUGH to Execute a Bulk Fetch - Use DBMS_HS_PASSTHROUGH to Execute a Bulk Fetch](#)

[TRIGGER CREATION USING LINKED SERVER TABLE ON MS SQL - Asked](#)

Los resultados de búsqueda proporcionan información útil. El resultado 4 muestra que `dbms_utility.exec_ddl_statement` se puede usar para DDL remoto. El resultado 5 muestra que `DBMS_HS_PASSTHROUGH.EXECUTE_IMMEDIATE` se puede usar para DDL y otros comandos. El resultado 7 indica que no se puede crear un trigger en una base de datos remota. Ahora necesito formular la respuesta. La respuesta debe incluir: 1. El procedimiento completo con UPDATE e INSERT en lugar de MERGE. 2. Una explicación de cómo crear la tabla y el trigger desde Oracle usando el database link. Para la creación de la tabla, se puede usar `DBMS_HS_PASSTHROUGH.EXECUTE_IMMEDIATE`. Para el trigger, no se puede crear directamente desde Oracle; debe crearse en SQL Server. La respuesta debe ser detallada y paso a paso, como para un niño de 12 años.

Entendido. Vamos a reemplazar el `MERGE` por un `UPDATE` y un `INSERT` separados, que es más fácil de depurar y no tiene las limitaciones del `MERGE` a través de un gateway. También te explico cómo crear la tabla y el trigger desde Oracle usando tu database link existente.

Procedimiento de Sincronización con `UPDATE` e `INSERT`

El siguiente procedimiento en Oracle reemplaza al `MERGE`. Primero actualiza los registros que ya existen en SQL Server y luego inserta los que faltan. Es más largo, pero mucho más claro y controlable.

sql

```
CREATE OR REPLACE PROCEDURE PR_SINC_BLOQUEOS_A_SQL AS
    v_filas_actualizadas NUMBER := 0;
    v_filas_insertadas   NUMBER := 0;
BEGIN
    -- PASO 1: ACTUALIZAR Los registros que YA EXISTEN en SQL Server.
    -- Se actualizan todos Los puntos de venta bloqueados en Oracle.
    -- La condición EXISTS asegura que solo se actualicen Los que ya están en
    SQL Server.
    UPDATE "tbbloqueopv"@mssqldb t
        SET t.motivo = (SELECT s.motivo
                        FROM tb_bloq_p_v s
                        WHERE s.id_punto_venta = t.id_punto_venta
                        AND s.estado = 'INACTIVO'),
        t.fecha_bloqueo = SYSDATE
    WHERE EXISTS (SELECT 1
                  FROM tb_bloq_p_v s
                  WHERE s.id_punto_venta = t.id_punto_venta
                  AND s.estado = 'INACTIVO');

    v_filas_actualizadas := SQL%ROWCOUNT;
    DBMS_OUTPUT.PUT_LINE('Registros actualizados en SQL Server: ' ||
v_filas_actualizadas);

    -- PASO 2: INSERTAR Los registros que NO EXISTEN en SQL Server.
    -- Se insertan todos Los puntos de venta bloqueados en Oracle
    -- que no tengan su ID en La tabla remota.
```

```

INSERT INTO "tbbloqueopv"@mssqldb
    (id_punto_venta, motivo, fecha_bloqueo)
SELECT s.id_punto_venta,
       s.motivo,
       SYSDATE
FROM   tb_bloq_p_v s
WHERE  s.estado = 'INACTIVO'
      AND NOT EXISTS (SELECT 1
                      FROM "tbbloqueopv"@mssqldb t
                      WHERE t.id_punto_venta = s.id_punto_venta);

v_filas_insertadas := SQL%ROWCOUNT;
DBMS_OUTPUT.PUT_LINE('Registros insertados en SQL Server: ' ||
v_filas_insertadas);

-- Confirmar la transacción distribuida.
COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error en sincronización: ' || SQLERRM);
        RAISE;

END;
/

```

Puntos clave del procedimiento:

- **UPDATE primero, INSERT después:** Es el orden lógico para evitar duplicados. Si se hiciera al revés, el `INSERT` podría fallar por violación de clave primaria.
- **WHERE EXISTS en el UPDATE:** Asegura que solo se actualicen las filas de SQL Server que corresponden a un punto de venta bloqueado en Oracle. Es más eficiente que actualizar toda la tabla remota.
- **NOT EXISTS en el INSERT:** Asegura que solo se inserten los IDs que no existen en la tabla remota. Evita el error de "duplicate key".
- **SQL%ROWCOUNT:** Te permite saber cuántas filas se vieron afectadas en cada operación. Es muy útil para el log y para diagnosticar problemas.
- **COMMIT al final:** Confirma la transacción distribuida. Si algo falla antes, el `ROLLBACK` en el `EXCEPTION` deshace todo.

🔧 ¿Puedo crear la tabla y el trigger desde Oracle?

La respuesta corta es: **la tabla sí, el trigger no**. Te explico por qué.

Crear la tabla en SQL Server desde Oracle: Sí se puede

Puedes usar el paquete `DBMS_HS_PASSTHROUGH` para enviar comandos DDL (como `CREATE TABLE`) directamente a SQL Server a través de tu database link. La sintaxis es:

sql

```
DECLARE
    v_resultado INTEGER;
BEGIN
    v_resultado := DBMS_HS_PASSTHROUGH.EXECUTE_IMMEDIATE@msqldb(
        'CREATE TABLE dbo.tbbloqueopv (
            id_punto_venta INT NOT NULL PRIMARY KEY,
            motivo VARCHAR(200) NULL,
            fecha_bloqueo DATETIME NOT NULL DEFAULT GETDATE()
        )'
    );
    DBMS_OUTPUT.PUT_LINE('Tabla creada. Filas afectadas: ' || v_resultado);
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error creando tabla: ' || SQLERRM);
        RAISE;
END;
/
```

Requisito importante: El usuario de SQL Server configurado en tu database link debe tener el privilegio `CREATE TABLE` en la base de datos de destino. Si no lo tiene, el comando fallará.

Crear el trigger en SQL Server desde Oracle: No se puede

Aquí está la limitación clave: **no se puede crear un trigger en una base de datos remota a través de un database link** que use un gateway heterogéneo.

Un trigger es un objeto de base de datos que debe residir **físicamente** en la misma base de datos que la tabla que lo dispara. El gateway de Oracle traduce consultas DML (SELECT, INSERT, UPDATE, DELETE), pero no permite crear objetos procedurales como triggers en el lado de SQL Server.

La única forma de crear el trigger es conectarte directamente a SQL Server (con SQL Server Management Studio o cualquier cliente SQL Server) y ejecutar el script `CREATE TRIGGER` allí. No hay forma de evitarlo.

Resumen de lo que puedes y no puedes hacer desde Oracle

Acción	¿Se puede hacer desde Oracle con @mssqldb?	Cómo
Crear tabla	✓ Sí	DBMS_HS_PASSTHROUGH.EXECUTE_IMMEDIATE
Insertar/Actualizar datos	✓ Sí	INSERT, UPDATE directamente
Crear trigger	✗ No	Debe hacerse conectándose directamente a SQL Server
Crear índice	✓ Sí	DBMS_HS_PASSTHROUGH.EXECUTE_IMMEDIATE
Crear procedimiento almacenado	✓ Sí (en algunos casos)	DBMS_HS_PASSTHROUGH.EXECUTE_IMMEDIATE con CREATE PROCEDURE

Recomendación Final

1. **Usa el procedimiento con UPDATE + INSERT** que te di arriba. Es más robusto y fácil de mantener que el MERGE.
2. **Crea la tabla tbbloqueopv desde Oracle** usando DBMS_HS_PASSTHROUGH. Así no necesitas pedirle permiso a nadie para crearla.
3. **Crea el trigger directamente en SQL Server.** Pídele acceso a un DBA de SQL Server o conéctate tú mismo con SQL Server Management Studio. No hay otra forma.
4. **Prueba todo en un entorno de desarrollo** antes de tocar producción. Ejecuta el procedimiento de sincronización, verifica que los datos lleguen a tbbloqueopv, y luego intenta insertar una venta para ese punto de venta. El trigger debe lanzar el error y bloquear la venta.

Con esto tienes todo lo necesario para implementar la solución completa. Si te surge algún error específico al ejecutar el DBMS_HS_PASSTHROUGH, pégamelo y lo resolvemos.

como hago para ver la configuracion de conexion que tiene el link o gateway que ya se usa en oracle llamado @mssqldb, para poder conectarme usando mss server managment y crear la tabla y el trigger en la db

El usuario quiere saber cómo ver la configuración de conexión del database link @mssqldb en Oracle para poder conectarse a SQL Server con SQL Server Management Studio (SSMS) y crear la tabla y el trigger manualmente.

Necesito proporcionar una guía paso a paso para obtener los detalles de conexión desde Oracle. Esto implica consultar las vistas del diccionario de datos de Oracle que almacenan información sobre los database links, como ALL_DB_LINKS, DBA_DB_LINKS o USER_DB_LINKS, y también verificar la configuración del gateway heterogéneo (por ejemplo, el archivo initdg4msql.ora, listener.ora, tnsnames.ora, y las configuraciones de ODBC).

Para ello, realizaré varias búsquedas que cubran los aspectos clave de la obtención de la configuración de conexión de un database link de Oracle a SQL Server a través de un gateway heterogéneo.

Found 44 web pages

Los resultados de búsqueda proporcionan información sobre las vistas del diccionario de datos de Oracle para database links, la configuración del gateway heterogéneo y los archivos de configuración de red. Para obtener una guía completa, es necesario abrir varios de estos resultados.

Read 5 pages

For example, you can create and execute the script below to determine which links are open (sample output included):

<table><tr><td>ビュー</td><td>用途</td></tr><tr><td>DBA DB LINKS</td><td>データベース内のデータベース・リンクがすべてリストされます

<table><tr><td>Variable</td><td>Description</td></tr><tr><td>service_name</td><td>specifies the service name for the gateway SID...

The gateway system identifier (SID) is an alphanumeric character string that identifies a gateway instance

SID LIST LISTENER= (SID LIST= (SID DESC=

Ahora tengo información sobre cómo obtener los detalles de conexión de un database link de Oracle a SQL Server a través de un gateway heterogéneo. Los resultados de búsqueda proporcionan información sobre las vistas del diccionario de datos, la función RESOLVE_TNSNAME y los archivos de configuración del gateway. Con esta información, puedo construir una guía paso a paso para el usuario. La guía debe cubrir: consultar las vistas del diccionario de datos para obtener el nombre del link y el host, usar DBMS_TNS.RESOLVE_TNSNAME para resolver el host, localizar y revisar el archivo tnsnames.ora, encontrar el archivo init<sid>.ora del gateway, obtener los detalles de conexión de ODBC/DSN, y finalmente conectarse a SQL Server con SSMS para crear la tabla y el trigger. La respuesta debe ser clara y detallada, y explicar por qué se necesita cada paso. Para conectarte a SQL Server con SSMS y crear los objetos que faltan, necesitas obtener los detalles de conexión que Oracle ya tiene configurados para el database link @mssqlldb. Esto se hace consultando el diccionario de datos de Oracle y revisando los archivos de configuración del gateway.

Paso 1: Obtener el nombre del Database Link y el Host desde Oracle

Conéctate a tu base de datos Oracle con un usuario que tenga privilegios (por ejemplo, `SYSTEM` o `SYS`) y ejecuta la siguiente consulta. Esto te dará el nombre exacto del link y el alias de red (TNS) que usa para conectarse al gateway.

sql

```
SELECT db_link, username, host
```

```
FROM all_db_links
WHERE db_link LIKE '%MSSQLDB%';
```

- **DB_LINK**: Te confirmará el nombre exacto (probablemente `MSSQLDB.REGRESS.RDBMS.DEV.US.Oracle.COM` o similar).
- **USERNAME**: El usuario con el que Oracle se conecta al gateway. A menudo es un usuario de Oracle, no de SQL Server directamente.
- **HOST**: Este es el **alias TNS** que apunta a la configuración del gateway. Anota este valor (ej. `MSSQL_GATEWAY`), lo usarás en el siguiente paso.

Paso 2: Resolver el Host a la cadena de conexión real

El campo **HOST** del link suele ser un alias. Para obtener la cadena de conexión completa (con la IP y el puerto del servidor donde está el gateway), usa el paquete `DBMS_TNS`:

sql

```
SELECT db_link, DBMS_TNS.RESOLVE_TNSNAME(host) AS connect_string
FROM all_db_links
WHERE db_link LIKE '%MSSQLDB%';
```

El resultado (`CONNECT_STRING`) te mostrará algo como `(DESCRIPTION=(ADDRESS=(PROTOCOL=TCP)(HOST=192.168.1.50)(PORT=1521))(CONNECT_DATA=(SID=dg4msql)))`. Ese **HOST** y **PORT** son los del servidor donde está instalado el Oracle Database Gateway, no los de SQL Server directamente.

Paso 3: Localizar el archivo `tnsnames.ora` de Oracle

El alias TNS que anotaste en el Paso 1 (**HOST** del link) está definido en el archivo `tnsnames.ora` de tu base de datos Oracle. Necesitas abrirlo para ver la configuración completa.

1. Ruta típica del archivo:

- En Windows: `%ORACLE_HOME%\network\admin\tnsnames.ora` (ej. `C:\oracle\Middleware\Oracle_Home\network\admin\tnsnames.ora`).
- Si no lo encuentras, busca la variable de entorno `TNS_ADMIN` en tu sistema; esa variable apunta a la carpeta correcta.

2. **Abre el archivo con el Bloc de Notas** y busca el alias que anotaste (ej. `MSSQL_GATEWAY`). Verás una entrada similar a esta:

text

```
MSSQL_GATEWAY =  
  (DESCRIPTION =  
    (ADDRESS = (PROTOCOL = TCP)(HOST = 192.168.1.50)(PORT = 1521))  
    (CONNECT_DATA = (SID = dg4msql))  
    (HS = OK)  
  )
```

- **HOST y PORT:** Corresponden al servidor donde está instalado el **Oracle Gateway** (no a SQL Server). Anótalos.
- **SID:** Es el identificador del gateway (ej. `dg4msql`). Anótalo.

Paso 4: Encontrar la configuración real de conexión a SQL Server (archivo `init<sid>.ora`)

Ahora necesitas encontrar el archivo de inicialización del gateway, que contiene la información real de conexión a SQL Server.

1. **Localiza el Oracle Home del Gateway.** Es un directorio separado del Oracle Home de la base de datos. Busca una carpeta llamada `dg4msql` dentro de un Oracle Home (ej. `C:\oracle\product\12.2.0\tg_1` o `C:\oracle\Middleware\Oracle_Home\dg4msql`). La ruta exacta depende de cómo se instaló el gateway.
2. **Abre el archivo `init<SID>.ora`.** El nombre del archivo incluye el SID que anotaste en el Paso 3. Por ejemplo, si el SID es `dg4msql`, busca:

text

```
[RUTA_ORACLE_HOME_GATEWAY]\dg4msql\admin\initdg4msql.ora
```

Ábrelo con el Bloc de Notas. El contenido será similar a esto:

text

```
HS_FDS_CONNECT_INFO = MSSQL_DSN  
HS_FDS_TRACE_LEVEL = OFF
```

- **HS_FDS_CONNECT_INFO:** Este es el **nombre del Origen de Datos ODBC (DSN)** configurado en el servidor del gateway. Ese DSN es el que realmente sabe cómo

conectarse a SQL Server (IP, puerto, base de datos, usuario, contraseña). Anótalo (ej. MSSQL_DSN).

Paso 5: Obtener los datos de conexión de ODBC

Si no tienes el archivo `initdg4msql.ora` a la mano o quieres confirmar los datos, necesitas acceder al servidor donde está instalado el gateway y revisar la configuración de ODBC.

1. **En el servidor del gateway**, abre el **Administrador de orígenes de datos ODBC** (busca "ODBC" en el menú de inicio de Windows).
2. Ve a la pestaña **"DSN de sistema"** (System DSN) y busca el DSN que anotaste (ej. MSSQL_DSN).
3. **Selecciónalo y haz clic en "Configurar"**. Ahí verás:
 - **Nombre del servidor (Server)**: La IP o nombre del servidor SQL Server.
 - **Base de datos (Database)**: El nombre de la base de datos de destino.
 - **Usuario (User)**: El usuario de SQL Server que se usa para la conexión.

Con estos datos (servidor, base de datos y usuario) ya tienes todo lo necesario para conectarte con SSMS. La contraseña la debe proporcionar el DBA de SQL Server o el administrador del gateway.

Paso 6: Conectarte con SQL Server Management Studio (SSMS)

Una vez que tengas los datos del Paso 5, abre SSMS y conéctate:

1. **Server type**: Database Engine.
2. **Server name**: La IP o nombre del servidor SQL Server que obtuviste del DSN.
3. **Authentication**: SQL Server Authentication (o Windows Authentication si el DSN usa autenticación integrada).
4. **Login y Password**: El usuario y contraseña de SQL Server (pídeselos a tu DBA si no los tienes).
5. Haz clic en **Connect**.

Una vez conectado, expande **Databases**, busca la base de datos correcta, haz clic derecho en **Tables** → **New** → **Table** y crea la tabla `tbbloqueopv` con el diseño que ya definimos. Para el trigger, abre una **New Query**, pega el script `CREATE TRIGGER` que te di y ejecútalo (asegurándote de estar en la base de datos correcta).

Resumen de los datos que necesitas

Dato	Dónde lo obtienes
Servidor SQL Server (IP/Nombre)	Paso 5 (Propiedades del DSN ODBC)
Base de datos SQL Server	Paso 5 (Propiedades del DSN ODBC)
Usuario SQL Server	Paso 5 (Propiedades del DSN ODBC)
Contraseña SQL Server	Pídesela al DBA de SQL Server

Con estos datos ya puedes conectarte con SSMS y crear los objetos que faltan. Si en algún paso no encuentras un archivo o no tienes acceso al servidor del gateway, pídele ayuda al equipo de TI; ellos deberían tener esta información documentada.

ahora generame una sola guía con el detalle paso a paso de la opción que vamos a usar para resolver el problema y también agrega los comandos o la forma de ver la conexión del link o gateway existente que ya usamos en varios procesos de Oracle

El usuario necesita una guía completa paso a paso para implementar la solución de sincronización de bloqueos de puntos de venta desde Oracle hacia SQL Server, incluyendo cómo verificar la configuración del database link existente `@mssqladb`. La guía debe ser detallada, con el código completo (UPDATE e INSERT en lugar de MERGE) y los comandos para inspeccionar la conexión del link/gateway.

Para proporcionar una guía precisa, necesito buscar información sobre cómo verificar la configuración de un database link de Oracle hacia SQL Server, incluyendo consultas al diccionario de datos y archivos de configuración del gateway. También es útil confirmar la sintaxis de `DBMS_HS_PASSTHROUGH` para crear tablas desde Oracle. Además, el usuario mencionó que ya usan varios procesos con el link, así que la información sobre la inspección del link es crucial.

Voy a realizar búsquedas para cubrir estos aspectos técnicos.

Found 37 web pages

Los resultados de las búsquedas proporcionan información útil. El resultado 0 muestra cómo usar `DBMS_TNS.RESOLVE_TNSNAME` para obtener el host de un database link. El resultado 1 explica la configuración del gateway y el archivo `initdg4msql.ora`. El resultado 2 confirma que `DBMS_HS_PASSTHROUGH.EXECUTE_IMMEDIATE` se puede usar para crear tablas en SQL Server. El resultado 3 muestra ejemplos de INSERT/UPDATE a través del gateway. Con esta información, puedo construir una guía completa. Ahora procederé a redactar la respuesta final.

Entendido. Vamos a consolidar todo en una única guía paso a paso para implementar la sincronización de bloqueos desde Oracle hacia SQL Server, usando el database link `@mssql1db` que ya tienes configurado. La guía cubre desde la verificación de la conexión existente hasta la creación del trigger en SQL Server.

Objetivo

Replicar la lógica de bloqueo de puntos de venta en SQL Server para que el sistema que consume esa base de datos también impida emitir ventas cuando un punto de venta está bloqueado en Oracle.

Paso 1: Verificar la conexión del database link **`@mssql1db`**

Antes de crear objetos, verifica que el link funciona correctamente y obtén los datos de conexión reales a SQL Server. Ejecuta estas consultas en Oracle (como usuario con privilegios, ej. `SYSTEM`):

1.1 Confirmar que el link existe y funciona

sql

```
-- Probar la conexión básica
SELECT * FROM DUAL@mssqldb;

-- Ver los detalles del link en el diccionario de datos
SELECT db_link, username, host
      FROM all_db_links
     WHERE db_link LIKE '%MSSQLDB%';
```

El campo `HOST` te dará el alias TNS que usa Oracle para llegar al gateway. Anótalo.

1.2 Resolver el alias TNS a la cadena de conexión real

Usa el paquete `DBMS_TNS` para obtener la dirección IP y puerto del servidor donde está instalado el gateway:

sql

```
SELECT db_link,
       DBMS_TNS.RESOLVE_TNSNAME(host) AS connect_string
      FROM all_db_links
     WHERE db_link LIKE '%MSSQLDB%';
```

El resultado será algo similar a:

text

```
(DESCRIPTION=(ADDRESS=(PROTOCOL=TCP)(HOST=192.168.1.50)(PORT=1521))
(CONNECT_DATA=(SID=dg4msql)))
```

Los valores de `HOST` y `PORT` corresponden al servidor del **Oracle Database Gateway**, no a SQL Server directamente. Anótalos.

1.3 Encontrar la configuración real de conexión a SQL Server

El gateway usa un archivo de inicialización `init<sid>.ora` que contiene la información real de conexión. Este archivo está en el Oracle Home del gateway, en una ruta similar a:

text

```
[RUTA_ORACLE_HOME_GATEWAY]\dg4msql\admin\initdg4msql.ora
```

Ábrelo con el Bloc de Notas. Verás algo como:

text

```
HS_FDS_CONNECT_INFO = MSSQL_DSN
HS_FDS_TRACE_LEVEL = OFF
```

El valor `HS_FDS_CONNECT_INFO` es el nombre del **Origen de Datos ODBC (DSN)** configurado en el servidor del gateway. Ese DSN contiene la IP del servidor SQL Server, el nombre de la base de datos y el usuario de conexión.

1.4 Obtener los datos de ODBC (si es necesario)

Si no tienes acceso al archivo `initdg4msql.ora`, accede al servidor del gateway y abre el **Administrador de orígenes de datos ODBC** de Windows. Busca el DSN en la pestaña **"DSN de sistema"**, selecciónalo y haz clic en **"Configurar"**. Ahí verás:

- **Servidor (Server):** IP o nombre del servidor SQL Server
- **Base de datos (Database):** nombre de la base de datos
- **Usuario (User):** usuario de SQL Server

Con estos datos ya puedes conectarte con SQL Server Management Studio (SSMS). La contraseña la debes obtener del DBA de SQL Server.



Paso 2: Crear la tabla `tbbloqueopv` en SQL Server

Conéctate a SQL Server con SSMS usando los datos del Paso 1.4 y ejecuta:

sql

-- Ejecutar en SQL Server (en la base de datos del sistema de ventas)

```
CREATE TABLE dbo.tbbloqueopv (  
    id_punto_venta INT NOT NULL PRIMARY KEY,  
    motivo VARCHAR(200) NULL,  
    fecha_bloqueo DATETIME NOT NULL DEFAULT GETDATE()  
);
```

Alternativa: También puedes crear esta tabla desde Oracle usando `DBMS_HS_PASSTHROUGH.EXECUTE_IMMEDIATE@mssqldb`, siempre que el usuario de SQL Server tenga privilegio `CREATE TABLE`.

sql

-- Desde Oracle (opcional)

```
DECLARE  
    v_resultado INTEGER;  
BEGIN  
    v_resultado := DBMS_HS_PASSTHROUGH.EXECUTE_IMMEDIATE@mssqldb(  
        'CREATE TABLE dbo.tbbloqueopv (  
            id_punto_venta INT NOT NULL PRIMARY KEY,  
            motivo VARCHAR(200) NULL,  
            fecha_bloqueo DATETIME NOT NULL DEFAULT GETDATE()  
        )'  
    );  
    COMMIT;  
END;  
/
```

Paso 3: Crear el procedimiento de sincronización en Oracle

Este procedimiento reemplaza el `MERGE` por un `UPDATE` y un `INSERT` separados, lo cual es más fácil de depurar y no tiene las limitaciones del `MERGE` a través de un gateway.

sql

```
CREATE OR REPLACE PROCEDURE PR_SINC_BLOQUEOS_A_SQL AS
    v_filas_actualizadas NUMBER := 0;
    v_filas_insertadas   NUMBER := 0;
BEGIN
    -- PASO 1: ACTUALIZAR Los registros que YA EXISTEN en SQL Server
    UPDATE "tbbloqueopv"@mssqldb t
        SET t.motivo = (SELECT s.motivo
                        FROM tb_bloq_p_v s
                        WHERE s.id_punto_venta = t.id_punto_venta
                        AND s.estado = 'INACTIVO'),
        t.fecha_bloqueo = SYSDATE
    WHERE EXISTS (SELECT 1
                  FROM tb_bloq_p_v s
                  WHERE s.id_punto_venta = t.id_punto_venta
                  AND s.estado = 'INACTIVO');

    v_filas_actualizadas := SQL%ROWCOUNT;
    DBMS_OUTPUT.PUT_LINE('Registros actualizados en SQL Server: ' ||
v_filas_actualizadas);

    -- PASO 2: INSERTAR Los registros que NO EXISTEN en SQL Server
    INSERT INTO "tbbloqueopv"@mssqldb
        (id_punto_venta, motivo, fecha_bloqueo)
    SELECT s.id_punto_venta,
        s.motivo,
        SYSDATE
    FROM tb_bloq_p_v s
    WHERE s.estado = 'INACTIVO'
        AND NOT EXISTS (SELECT 1
                        FROM "tbbloqueopv"@mssqldb t
                        WHERE t.id_punto_venta = s.id_punto_venta);
```

```

        v_filas_insertadas := SQL%ROWCOUNT;
        DBMS_OUTPUT.PUT_LINE('Registros insertados en SQL Server: ' ||
v_filas_insertadas);

        COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error en sincronización: ' || SQLERRM);
        RAISE;
END;
/

```

Paso 4: Programar el Job en Oracle

El procedimiento debe ejecutarse **inmediatamente después** del proceso de bloqueo automático. Añade la llamada al final de tu procedimiento de bloqueo existente, o crea un Job independiente con un pequeño retraso:

sql

```

BEGIN
    DBMS_SCHEDULER.CREATE_JOB (
        job_name          => 'JOB_SINC_BLOQUEOS_SQL',
        job_type           => 'STORED_PROCEDURE',
        job_action         => 'PR_SINC_BLOQUEOS_A_SQL',
        start_date         => TRUNC(SYSDATE) + 6/24 + 5/1440, -- Hoy a las 6:05 AM
        repeat_interval    => 'FREQ=HOURLY; INTERVAL=2;
BYHOUR=6,8,10,12,14,16,18,20; BYMINUTE=5',
        enabled            => TRUE
    );
END;
/

```

Paso 5: Crear el trigger en SQL Server

Este trigger se dispara **antes** de insertar una venta. Si el punto de venta está en `tbbloqueopv`, lanza un error y no permite la inserción.

sql

```
-- Ejecutar en SQL Server (en la base de datos del sistema de ventas)
CREATE TRIGGER trg_validar_bloqueo_pv
ON dbo.ventas
INSTEAD OF INSERT
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (
        SELECT 1
        FROM inserted i
        JOIN dbo.tbbloqueopv b ON b.id_punto_venta = i.id_punto_venta
    )
    BEGIN
        THROW 50000, 'El punto de venta está bloqueado. No se puede emitir la
venta.', 1;
    END

    INSERT INTO dbo.ventas (id_venta, id_punto_venta, fecha_venta, monto)
    SELECT id_venta, id_punto_venta, fecha_venta, monto
    FROM inserted;
END;
```

Flujo completo del proceso

1. **Cada 2 horas**, el Job en Oracle ejecuta el proceso de bloqueo automático y actualiza `tb_bloq_p_v`.
2. **Inmediatamente después**, se ejecuta `PR_SINC_BLOQUEOS_A_SQL` que sincroniza los datos a `tbbloqueopv` en SQL Server vía `@mssqldb`.

3. Cuando el sistema de ventas intenta insertar una venta, el trigger `trg_validar_bloqueo_pv` verifica si el punto de venta está bloqueado.
4. Si está bloqueado, lanza el error `THROW` y la venta **no se inserta**.
5. La aplicación recibe el mensaje y lo muestra al usuario.

Resumen de objetos creados

Objeto	Dónde	Propósito
<code>tbbloqueopv</code>	SQL Server	Tabla espejo de puntos de venta bloqueados
<code>PR_SINC_BLOQUEOS_A_SQL</code>	Oracle	Procedimiento que sincroniza vía <code>@mssqldb</code>
<code>JOB_SINC_BLOQUEOS_SQL</code>	Oracle	Job que ejecuta la sincronización cada 2 horas
<code>trg_validar_bloqueo_pv</code>	SQL Server	Trigger que impide ventas en PV bloqueados

Consideraciones finales

- **Prueba en desarrollo:** Antes de producción, bloquea un PV en Oracle, ejecuta `PR_SINC_BLOQUEOS_A_SQL`, verifica que el registro llegue a `tbbloqueopv` y luego intenta insertar una venta en SQL Server. El trigger debe lanzar el error.
- **Manejo de errores:** El procedimiento en Oracle captura errores de red o de gateway. Si falla, el Job reintentará en la próxima ejecución.
- **Credenciales:** El usuario de SQL Server en el database link debe tener permisos de `SELECT`, `INSERT` y `UPDATE` sobre `tbbloqueopv`. Para crearla desde Oracle con `DBMS_HS_PASSTHROUGH`, también necesita `CREATE TABLE`.
- **Trigger en SQL Server:** No se puede crear desde Oracle con un database link. Debe ejecutarse directamente en SSMS.

Con esta guía tienes todo lo necesario para implementar la solución completa. Si surge algún error específico al ejecutar algún paso, compártelo y lo resolvemos.